

CQRS + MediatR Pattern - Complete Guide

Using Your Basket Project

Part 1: The Problem CQRS Solves

Imagine you have a simple controller without CQRS:

```
// WITHOUT CQRS - Everything in one place
public class BasketController : ControllerBase
{
    [HttpGet("{userName}")]
    public async Task<ActionResult> GetBasket(string userName)
    {
        // 1. Validation logic
        if (string.IsNullOrEmpty(userName))
            return BadRequest();

        // 2. Business logic
        var basket = await _repository.GetBasket(userName);

        // 3. Database logic
        // 4. Mapping logic
        // 5. Discount calculation
        // 6. Event publishing
        // ALL MIXED TOGETHER!
    }
}
```

Problems with this approach:

1. **Controller becomes huge** - Too much code in one place
2. **Hard to test** - Can't test business logic without HTTP
3. **Hard to reuse** - Can't use the same logic from another place
4. **Violates Single Responsibility** - Controller does too many things
5. **Read and Write mixed** - Reading data and writing data use same code path

CQRS Solution: Separate "Commands" (write operations) from "Queries" (read operations)

Part 2: Your Basket Project Structure

Your Basket project follows CQRS perfectly:

```
Basket.Application/  
├── Commands/           ← WRITE operations (change data)  
│   ├── CreateShoppingCartCommand.cs  
│   ├── DeleteBasketByUserNameCommand.cs  
│   └── BasketCheckoutCommand.cs  
├── Queries/           ← READ operations (get data)  
│   └── GetBasketByUserNameQuery.cs  
├── Handlers/          ← The logic for each command/query  
│   ├── CreateShoppingCartHandler.cs  
│   ├── DeleteBasketByUserNameHandler.cs  
│   ├── BasketCheckoutHandler.cs  
│   └── GetBasketByUserNameHandler.cs
```

Part 3: Step-by-Step Implementation Guide

Step 1: Install MediatR Package

Your file: Basket.Application/Basket.Application.csproj (line 18)

```
<PackageReference Include="MediatR" Version="14.0.0" />
```

What MediatR does: It's a messenger that sends commands/queries to the right handler. Think of it like a post office - you give it a letter (command), and it delivers it to the right person (handler).

Step 2: Register MediatR in Program.cs

Your file: Basket.API/Program.cs (lines 32-39)

```
// Register MediatR
var assemblies = new Assembly[]
{
    Assembly.GetExecutingAssembly(),
    typeof(CreateShoppingCartHandler).Assembly // This tells
MediatR where to find handlers
};
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssemblies(assemblies));
```

What this does:

- Scans your assemblies to find all handlers
- Automatically registers them with MediatR
- Now when you send a command, MediatR knows which handler to call

Step 3: Create a Query (READ Operation)

Your file: Basket.Application/Queries/GetBasketByUserNameQuery.cs

```
using Basket.Application.Responses;
using MediatR;

namespace Basket.Application.Queries
{
    public record GetBasketByUserNameQuery(string UserName) :
    IRequest<ShoppingCartResponse>;
}
```

Breakdown:

- record - A C# feature for immutable data (like a class but simpler)
- GetBasketByUserNameQuery - The name of your query
- string UserName - The input data (what you need to search)
- : IRequest<ShoppingCartResponse> - Tells MediatR this is a query that returns ShoppingCartResponse

Think of it like: A form you fill out - "I want to get basket for username 'john'"

Step 4: Create a Query Handler (The Logic)

Your file: Basket.Application/Handlers/GetBasketByUserNameHandler.cs

```
public class GetBasketByUserNameHandler :
    IRequestHandler<GetBasketByUserNameQuery, ShoppingCartResponse>
{
    private IBasketRepository _basketRepository;
    private ILogger<GetBasketByUserNameHandler> _logger;

    public GetBasketByUserNameHandler(IBasketRepository
basketRepository, ILogger<GetBasketByUserNameHandler> logger)
    {
        _basketRepository = basketRepository;
        _logger = logger;
    }

    public async Task<ShoppingCartResponse>
Handle(GetBasketByUserNameQuery request, CancellationToken
cancellation_token)
    {
        var shoppingCart = await
_basketRepository.GetBasket(request.UserName);
        if(shoppingCart == null)
        {
            return new ShoppingCartResponse(request.UserName)
            {
                Items = new List<ShoppingCartItemResponse>()
            };
        }
        _logger.LogInformation("Fetched basket for
{@UserName}", request.UserName);
        return shoppingCart.ToShoppingCartResponse();
    }
}
```

Breakdown:

- `IRequestHandler<GetBasketByUserNameQuery, ShoppingCartResponse>` - Implements the handler interface
- Constructor - Gets dependencies (repository, logger) via dependency injection
- Handle method - The actual logic:
 - Gets basket from Redis repository
 - If not found, returns empty basket
 - Logs the operation
 - Converts entity to response DTO

Think of it like: The worker who receives the form and does the actual work

Step 5: Create a Command (WRITE Operation)

Your file: `Basket.Application/Commands/CreateShoppingCartCommand.cs`

```
using Basket.Application.DTOS;
using Basket.Application.Responses;
using MediatR;

namespace Basket.Application.Commands
{
    public record CreateShoppingCartCommand(string UserName,
        List<CreateShoppingCartItemDto> items) :
        IRequest<ShoppingCartResponse>;
}
```

Difference from Query:

- Commands are for CHANGING data (create, update, delete)
- Queries are for READING data (get, search)
- Both look similar, but their purpose is different

Step 6: Create a Command Handler (The Logic)

Your file: `Basket.Application/Handlers/CreateShoppingCartHandler.cs`

```
public class CreateShoppingCartHandler :
    IRequestHandler<CreateShoppingCartCommand, ShoppingCartResponse>
{
```

```

private readonly IBasketRepository _basketRepository;
private readonly DiscountGrpcService _discountGrpcService;

public CreateShoppingCartHandler(IBasketRepository
basketRepository, DiscountGrpcService discountGrpcService)
{
    _basketRepository = basketRepository;
    _discountGrpcService = discountGrpcService;
}

public async Task<ShoppingCartResponse>
Handle(CreateShoppingCartCommand request, CancellationToken
cancellationToken)
{
    // Apply discounts using GRPC call
    foreach (var item in request.items)
    {
        var coupon = await
_discountGrpcService.GetDiscount(item.ProductName);
        item.Price = item.Price - coupon.Amount; // minus amount
from discount amount
    }
    // Convert command to domain entity
    var shoppingCartEntity = request.ToShoppingCartEntity();
    // Save to redis
    var updateCart = await
_basketRepository.UpsertBasket(shoppingCartEntity);
    // Convert back to response
    return updateCart.ToShoppingCartResponse();
}
}

```

What this handler does:

1. **Business logic:** Calls gRPC service to get discounts
2. **Calculation:** Subtracts discount from price
3. **Conversion:** Converts command to domain entity
4. **Persistence:** Saves to Redis

5. **Response:** Converts back to response DTO

Notice: Complex business logic is in the handler, NOT in the controller!

Step 7: Use in Controller (The Entry Point)

Your file: Basket.API/Controllers/BasketController.cs

```
[HttpGet("{userName}")]
public async Task<ActionResult<ShoppingCartDto>> GetBasket(string
userName)
{
    var query = new GetBasketByUserNameQuery(userName); // Create
query
    var result = await _mediator.Send(query);           // Send to
MediatR
    _logger.LogInformation("Fetching Basket for {@userName}",
userName);
    return Ok(result);
}

[HttpPost]
public async Task<ActionResult<ShoppingCartDto>>
CreateOrUpdateBasket([FromBody] CreateShoppingCartCommand
createShoppingCartCommand)
{
    var result = await _mediator.Send(createShoppingCartCommand); //
Send to MediatR
    return Ok(result);
}
```

What the controller does:

1. Creates the command/query object
2. Sends it to MediatR via `_mediator.Send()`
3. MediatR finds the right handler
4. Handler executes the logic
5. Result returns to controller
6. Controller returns HTTP response

Controller is now thin - just HTTP stuff, no business logic!

Part 4: The Complete Flow (Visual)

```
User Request
  ↓
Controller (BasketController.cs)
  ↓ creates
Query/Command (GetBasketByUserNameQuery.cs)
  ↓ sends to
MediatR (The Messenger)
  ↓ finds
Handler (GetBasketByUserNameHandler.cs)
  ↓ calls
Repository (BasketRepository.cs)
  ↓ reads from
Redis
  ↓ returns
Response
```

Part 5: Complex Example - Checkout Command

Your file: Basket.Application/Commands/BasketCheckoutCommand.cs

```
public record BasketCheckoutCommand(BasketCheckoutDto
BasketCheckoutDto) : IRequest<Unit>;
```

Note: Returns Unit (like void) because checkout doesn't return data - it just performs an action.

Your file: Basket.Application/Handlers/BasketCheckoutHandler.cs

```
public async Task<Unit> Handle(BasketCheckoutCommand request,
Cancellation token cancellationToken)
{
    var basketDto = request.BasketCheckoutDto;
```

```

    // 1. Get the basket (using MediatR to call another query!)
    var basketResponse = await _mediator.Send(new
GetBasketByUserNameQuery(basketDto.UserName), cancellationToken);

    if (basketResponse is null || !basketResponse.Items.Any())
    {
        throw new InvalidOperationException("Basket not found or
empty");
    }

    // 2. Convert to entity
    var basket = basketResponse.ToEntity();

    // 3. Map to event
    var evt = basketDto.ToBasketCheckoutEvent(basket);

    // 4. Publish event to RabbitMQ
    _logger.LogInformation("Publishing BasketCheckoutEvent For
{User}", basket.UserName);
    await _publishEndpoint.Publish(evt, cancellationToken);

    // 5. Delete the basket (using MediatR to call another command!)
    await _mediator.Send(new
DeleteBasketByUserNameCommand(basket.UserName), cancellationToken);

    return Unit.Value;
}

```

This shows the power of CQRS + MediatR:

- Handler can call other handlers via MediatR
- Clean separation of concerns
- Each handler does one thing well
- Easy to test each piece independently

Part 6: Benefits You Get

1. Clean Controllers

- Before: Controller with 100+ lines of logic
- After: Controller with 3-5 lines (just calls MediatR)

2. Easy Testing

- Test handlers without HTTP
- Mock dependencies easily
- Unit test business logic in isolation

3. Reusability

- Same handler can be called from:
 - HTTP API
 - gRPC service
 - Background worker
 - Another handler

4. Separation of Concerns

- Controller: HTTP concerns
- Handler: Business logic
- Repository: Data access

5. Scalability

- Can optimize reads and writes separately
- Can use different databases for read/write (advanced)

Part 7: Summary

CQRS = Command Query Responsibility Segregation

- **Commands:** Change data (Create, Update, Delete)
- **Queries:** Read data (Get, Search)

MediatR = The messenger

- Sends commands/queries to handlers
- Automatically finds the right handler
- Handles dependency injection

Your Implementation:

- ☒ Commands in Commands/ folder
- ☒ Queries in Queries/ folder
- ☒ Handlers in Handlers/ folder
- ☒ Clean controllers that just call MediatR
- ☒ Business logic in handlers
- ☒ Easy to test and maintain

This is professional-grade architecture used in enterprise applications!

I've completed the comprehensive guide on CQRS and MediatR pattern using your Basket project. The guide covers:

1. **The Problem** - Why traditional controllers become messy and hard to maintain
2. **Your Structure** - How your Basket project organizes Commands, Queries, and Handlers
3. **Step-by-Step Implementation** - From package installation to controller usage
4. **Complete Flow** - Visual diagram of how requests travel through the system
5. **Complex Example** - The checkout command showing how handlers can call other handlers
6. **Benefits** - Clean code, easy testing, reusability, separation of concerns

Your implementation follows professional CQRS patterns with:

- Commands for write operations (Create, Delete, Checkout)
- Queries for read operations (GetBasket)
- Handlers containing business logic
- Thin controllers that just delegate to MediatR

Is there any specific part of the CQRS pattern you'd like me to explain further, or would you like to see how this pattern is implemented in another service (like Catalog or Ordering)?